

Nama : Yudhistira Putra Hartanto

Kelas/Absen : 1G/29

NIM : 254107020083

Tugas Praktikum

Modifikasi

- Class BinaryTree29
package Pertemuan14;

public class BinaryTree29 {
 Node29 root;

 public BinaryTree29(){
 root = null;
 }

 public boolean isEmpty(){
 return root == null;
 }

 public void add(Mahasiswa29 mahasiswa) {
 Node29 newNode = new Node29(mahasiswa);
 if (isEmpty()) {
 root = newNode;
 } else {
 Node29 current = root;
 Node29 parent = null;
 while (true) {
 parent = current;
 if (mahasiswa.ipk < current.mahasiswa.ipk) {
 current = current.left;
 if (current == null) {
 parent.left = newNode;
 return;
 }
 } else {
 current = current.right;
 if (current == null) {
 parent.right = newNode;
 return;
 }
 }
 }
 }
 }

 boolean find(double ipk) {

```

boolean result = false;
Node29 current = root;
while (current != null) {
    if (current.mahasiswa.ipk == ipk) {
        result = true;
        break;
    } else if (ipk > current.mahasiswa.ipk) {
        current = current.right;
    } else {
        current = current.left;
    }
}
return result;
}

void traversePreOrder(Node29 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(Node29 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(Node29 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

Node29 getSuccessor(Node29 del) {
    Node29 successor = del.right;
    Node29 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

```

```

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }

    // cari node (current) yang akan dihapus
    Node29 parent = root;
    Node29 current = root;
    boolean isLeftChild = false;

    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // penghapusan
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    } else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            } else {
                if (isLeftChild) {
                    parent.left = null;
                } else {
                    parent.right = null;
                }
            }
        } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
            if (current == root) {
                root = current.right;
            } else {
                if (isLeftChild) {
                    parent.left = current.right;
                } else {
                    parent.right = current.right;
                }
            }
        } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
            if (current == root) {

```

```

        root = current.left;
    } else {
        if (isLeftChild) {
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    }
} else { // jika punya 2 anak
    Node29 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}

public void addRekursif(Mahasiswa29 mahasiswa) {
    root = addRekursif(root, mahasiswa);
}

private Node29 addRekursif(Node29 current, Mahasiswa29 mahasiswa) {
    if (current == null) {
        return new Node29(mahasiswa);
    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        current.right = addRekursif(current.right, mahasiswa);
    }

    return current;
}

public void cariMinIPK() {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node29 current = root;
    while (current.left != null) {
        current = current.left;
    }
    System.out.println("IPK terkecil: " + current.mahasiswa.ipk);
}

```

```

        current.mahasiswa.tampilInformasi();
    }

    public void cariMaxIPK() {
        if (isEmpty()) {
            System.out.println("Binary tree kosong");
            return;
        }
        Node29 current = root;
        while (current.right != null) {
            current = current.right;
        }
        System.out.println("IPK terbesar: " + current.mahasiswa.ipk);
        current.mahasiswa.tampilInformasi();
    }

    public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
        System.out.println("Mahasiswa dengan IPK di atas " + ipkBatas + ":");
        tampilMahasiswaIPKdiAtas(root, ipkBatas);
    }

    private void tampilMahasiswaIPKdiAtas(Node29 node, double ipkBatas) {
        if (node != null) {
            if (node.mahasiswa.ipk > ipkBatas) {
                node.mahasiswa.tampilInformasi();
            }
            tampilMahasiswaIPKdiAtas(node.left, ipkBatas);
            tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
        }
    }
}

```

- Class BinaryTreeMain29
package Pertemuan14;

```

public class BinaryTreeMain29 {
    public static void main(String[] args) {
        BinaryTree29 bst = new BinaryTree29();

        // bst.add(new Mahasiswa29("244160121", "Ali", "A", 3.57));
        // bst.add(new Mahasiswa29("244160221", "Badar", "B", 3.85));
        // bst.add(new Mahasiswa29("244160185", "Candra", "C", 3.21));
        // bst.add(new Mahasiswa29("244160220", "Dewi", "B", 3.54));

        // System.out.println("\nDaftar semua mahasiswa (in order traversal):");
        // bst.traverseInOrder(bst.root);

        // System.out.println("\nPencarian data mahasiswa:");
        // System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        // String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        // System.out.println(hasilCari);
    }
}

```

```

// System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
// hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
// System.out.println(hasilCari);

// bst.add(new Mahasiswa29("244160131", "Devi", "A", 3.72));
// bst.add(new Mahasiswa29("244160205", "Ehsan", "D", 3.37));
// bst.add(new Mahasiswa29("244160170", "Fizi", "B", 3.46));
// System.out.println("\nDaftar semua mahasiswa setelah penambahan 3
mahasiswa:");
// System.out.println("\nOrder Traversal:");
// bst.traverseInOrder(bst.root);
// System.out.println("\nPreOrder Traversal:");
// bst.traversePreOrder(bst.root);
// System.out.println("\nPostOrder Traversal:");
// bst.traversePostOrder(bst.root);

// System.out.println("\nPenghapusan data mahasiswa");
// bst.delete(3.57);
// System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");
// bst.traverseInOrder(bst.root);

bst.addRekursif(new Mahasiswa29("244160121", "Ali", "A", 3.57));
bst.addRekursif(new Mahasiswa29("244160221", "Badar", "B", 3.85));
bst.addRekursif(new Mahasiswa29("244160185", "Candra", "C", 3.21));
bst.addRekursif(new Mahasiswa29("244160220", "Dewi", "B", 3.54));
bst.addRekursif(new Mahasiswa29("244160131", "Devi", "A", 3.72));
bst.addRekursif(new Mahasiswa29("244160205", "Ehsan", "D", 3.37));
bst.addRekursif(new Mahasiswa29("244160170", "Fizi", "B", 3.46));

System.out.println("\nDaftar semua mahasiswa (InOrder Traversal):");
bst.traverseInOrder(bst.root);

System.out.println("\n=== HASIL TUGAS PRAKTIKUM ===");
bst.cariMinIPK();
bst.cariMaxIPK();

System.out.println();
bst.tampilMahasiswaIPKdiAtas(3.50);

}
}

```

- Class BinaryTreeArray29
package Pertemuan14;

```

public class BinaryTreeArray29 {
    Mahasiswa29[] dataMahasiswa;
    int idxLast;
    int size;

    public BinaryTreeArray29() {
        this.dataMahasiswa = new Mahasiswa29[10];
    }
}

```

```

    }

    void populateData(Mahasiswa29 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }

    void add(Mahasiswa29 data) {
        if (idxLast + 1 >= size) {
            // Resize array jika penuh
            Mahasiswa29[] newArray = new Mahasiswa29[size * 2];
            System.arraycopy(dataMahasiswa, 0, newArray, 0, size);
            dataMahasiswa = newArray;
            size = size * 2;
        }
        idxLast++;
        dataMahasiswa[idxLast] = data;
    }

    void traversePreOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                dataMahasiswa[idxStart].tampilInformasi();
                traversePreOrder(2 * idxStart + 1);
                traversePreOrder(2 * idxStart + 2);
            }
        }
    }
}

```

- Class BinaryTreeArrayMain29
package Pertemuan14;

```

public class BinaryTreeArrayMain29 {
    public static void main(String[] args) {
        BinaryTreeArray29 bta = new BinaryTreeArray29();

        Mahasiswa29 mhs1 = new Mahasiswa29("244160121", "Ali", "A", 3.57);
        Mahasiswa29 mhs2 = new Mahasiswa29("244160185", "Candra", "C", 3.41);
        Mahasiswa29 mhs3 = new Mahasiswa29("244160221", "Badar", "B", 3.75);
        Mahasiswa29 mhs4 = new Mahasiswa29("244160220", "Dewi", "B", 3.35);
        Mahasiswa29 mhs5 = new Mahasiswa29("244160131", "Devi", "A", 3.48);
        Mahasiswa29 mhs6 = new Mahasiswa29("244160205", "Ehsan", "D", 3.61);
    }
}

```

```

Mahasiswa29 mhs7 = new Mahasiswa29("244160170", "Fizi", "B", 3.86);

Mahasiswa29[] dataMahasiswa = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7,
null, null, null};
int idxLast = 6;
bta.populateData(dataMahasiswa, idxLast);

System.out.println("\nInorder Traversal Mahasiswa: ");
bta.traverseInOrder(0);

System.out.println("\n=== HASIL TUGAS PRAKTIKUM ===");
System.out.println("PreOrder Traversal Mahasiswa: ");
bta.traversePreOrder(0);
}
}

```

- Hasil Class BinaryTreeMain29

Daftar semua mahasiswa (InOrder Traversal):

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21
 NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.37
 NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.46
 NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54
 NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57
 NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72
 NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

=== HASIL TUGAS PRAKTIKUM ===

IPK terkecil: 3.21

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

IPK terbesar: 3.85

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

Mahasiswa dengan IPK di atas 3.5:

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57
 NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54
 NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85
 NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

- Hasil Class BinaryTreeArrayMain29

Inorder Traversal Mahasiswa:

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.35
 NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.41
 NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.48
 NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57
 NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.61
 NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.75
 NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.86

=== HASIL TUGAS PRAKTIKUM ===

PreOrder Traversal Mahasiswa:

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57
 NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.41
 NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.35

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.48
NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.75
NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.61
NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.86